

NeeruAI

A closed-loop water and heat operating system for India's data centres — built on MongoDB.

SUBMISSION

**Project & MongoDB architecture
report**

CATEGORY

**Climate / water infrastructure ·
RegTech**

DATE

September 2026

NeeruAI reduces a data centre's freshwater demand using closed liquid cooling, recovers water from its concentrated reject stream using the facility's own waste heat, and cryptographically verifies every recovered litre before it is reused or released.

India's data centres consume roughly **150 billion litres of water a year**, much of it drawn from aquifers already past their sustainable limit. No regulation requires a facility to publish what it actually withdraws or discharges, and no agency verifies that the water source named in a clearance matches what happens on site. That unverifiable gap is the problem this project addresses — and MongoDB is the reason the solution is buildable as a single coherent data layer rather than four separate systems stitched together.

§ Contents

01	Executive summary	What was built, and what is novel
02	The problem	Water, heat, and an absent verification regime
03	The solution — reduce, recover, verify	Three layers and the physical chain
04	System architecture	Ingestion, forecast, dispatch, verification
05	MongoDB data architecture	The core of this submission — 12 sub-sections
06	The prototype	Six pages, annotated with figures
07	Engineering model	The physics behind every number on screen
08	Roadmap to production	What a real deployment needs next
09	Sources & honest limitations	What is modelled, what is measured

At a glance

169%

Groundwater extraction at Devanahalli, against the sustainable limit
CGWB

150 bn L

Annual water use by Indian data centres
CEEW 2026

0

Facilities required to publish actual consumption or discharge
No rule exists

6

MongoDB capabilities doing load-bearing work in the design
§05

5

Collections, one of them a native time-series collection
§5.2

69 M

Telemetry documents per day at a 100-facility deployment
§5.11

01 Executive summary

NeeruAI is a software layer — not a treatment plant. It makes water-recovery hardware that a data centre already owns, or could buy from any vendor, coordinated, forecastable and independently auditable.

The three layers

LAYER	WHAT IT DOES	MECHANISM
01 REDUCE	Cuts the freshwater draw at its source	Closed-loop liquid cooling — direct-to-chip or immersion — instead of open evaporative towers. The coolant recirculates; it is not consumed.
02 RECOVER	Takes back what is still lost	Waste heat drives membrane distillation on the blowdown / RO-reject stream: already concentrated, already contaminated, today simply drained.
03 VERIFY	Proves it happened	Inline probes → automated quality gate against BIS IS 10500 <i>drinking-water</i> limits → hash-chained, independently recomputable ledger.

Why this is a software problem

Recoverable heat is a function of IT load, and IT load in India's mixed air- and liquid-cooled fleet is volatile and site-specific. A fixed hardware topology cannot absorb that variance. A forecasting and dispatch layer can — and it works with whatever treatment asset a facility already runs, whether that is a reverse-osmosis skid, an existing effluent treatment plant, a municipal ozonation unit, or a future membrane distillation module.

The nearest prior art, a 2018 patent covering waste-heat-driven low-temperature desalination, claims a *hardware method* on one steady liquid-cooled loop with a seawater feed. It claims nothing about monitoring, forecasting, dispatch or verification, and its assumptions do not survive contact with an inland Indian facility whose water source is undisclosed and whose heat availability swings with a training run.

What was built

- A **six-page operations console** that simulates the entire plant from one physics chain: IT load → heat → coolant temperature → membrane flux → permeate chemistry → pass/fail → dispatch → statutory report. Changing any control on one page moves every downstream page consistently, because there is exactly one model underneath.
- A **pore-scale membrane distillation simulation** in which water vapour crosses hydrophobic pores while fluoride, nitrate and dissolved-salt particles are rejected — with flux computed from the Antoine vapour-pressure relation and corrected for temperature polarisation.
- A **hash-chained verification ledger** with a working independent audit, plus a fault-injection control that breaches the membrane and demonstrates the quality gate quarantining a non-compliant batch in real time.
- A **MongoDB data architecture** covering ingestion, forecasting, reactive dispatch, geospatial prioritisation, compliance reporting and ledger integrity — detailed in §05, which is the substance of this submission.

THE MONGODB THESIS IN ONE PARAGRAPH

This system is four workloads that normally live in four different databases: a high-frequency IoT stream, a polymorphic device-integration layer, a geospatial routing problem, and an append-only audit log with a regulatory reporting API on top. MongoDB collapses all four into one deployment — time-series collections for the stream, flexible documents for the polymorphism, `2dsphere` for the routing, and a hash-chained document field for the audit log, with change streams tying them together reactively and the aggregation framework producing the regulator's report. Removing any one of those capabilities does not degrade the product; it removes a specific feature.

02 The problem

2.1 The aquifer is already past the line

FIGURE	MEANING	SOURCE
169%	Devanahalli's groundwater extraction as a share of its permissible limit — no headroom remains	CGWB
1,000 ft	Typical borewell depth in that belt; natural recharge reaches only about 200 ft	Field reporting
60.47%	India's national stage of groundwater extraction; 11.1% of assessment units over-exploited	CGWB 2024
104.79%	Gautam Buddha Nagar (Noida) — evidence this is a national pattern, not a Bengaluru story	CGWB 2024
1.5 mg/L	WHO fluoride limit for drinking water; hard-rock aquifers in this geology run well above it	WHO; Springer 2026
~150 bn L	Annual water use by Indian data centres, on roughly 0.5% of national electricity	CEEW 2026
20% → 50%	Mandatory wastewater reuse targets for large water users, 2027–28 rising to 2030–31	Liquid Waste Management Rules, 2024

The water being drawn is not merely scarce; it is contaminated. **Fluoride** is geogenic, released from deep hard-rock aquifers, and causes skeletal fluorosis. **Nitrate** arrives from agricultural and sewage infiltration and exceeds the 45 mg/L limit across a large share of shallow Bengaluru wells. Both are non-volatile — which, as §3.3 explains, is precisely why the treatment technology chosen here works.

2.2 The seawater question is a false binary

Public debate frames this as "seawater cooling versus groundwater". That describes a choice that does not yet exist. Seawater cooling in India is **proposed, not deployed**: Andhra Pradesh's data centre policy floats it for the Tarluvada hyperscale park, but the environmental clearance for that project *names no water source at all*, and monitoring groups cannot establish whether it will draw desalinated, surface or ground water.

The real, present contest is between **groundwater, municipal supply, treated non-potable water, and recycled blowdown** — and almost no operator transparently discloses which they use:

- Nxtra by Airtel's 2025 ESG report: 216,357 KL total consumption — 9,876 KL groundwater, 191,152 KL purchased third-party, zero surface water.
- A Greater Noida operator in a notified over-exploited zone claims it uses neither groundwater nor municipal drinking water, while officials would not confirm whether it holds extraction permission at all.
- A second Greater Noida facility runs entirely on the industrial authority's treated non-potable network — the correct model, and one worth evidencing publicly so others copy it.

Nor is seawater automatically benign: thermal discharge, intake entrainment, seabed disturbance from pipe runs, biofouling chemistry and desalination brine are all live impacts. The defensible position is that **seawater can reduce freshwater demand, but moving the cooling problem from an aquifer to the ocean does not by itself create a closed loop**.

2.3 The regulatory gap that defines the product

Groundwater extraction requires a No Objection Certificate specifying a withdrawal cap. Beyond that:

- Companies are **not required to publish** actual consumption or discharge figures. The EU mandates disclosure; India does not.

- **Once a clearance is granted, no agency verifies** that the declared water source matches on-site practice.
- Karnataka's 2022 Data Centre Policy required no water sustainability report whatsoever.

The consequence is that corporate "water-neutral" claims are unfalsifiable. Without independent verification they are indistinguishable from greenwashing — and that is the hole a verification layer fills.

2.4 What actually happens to cooling water

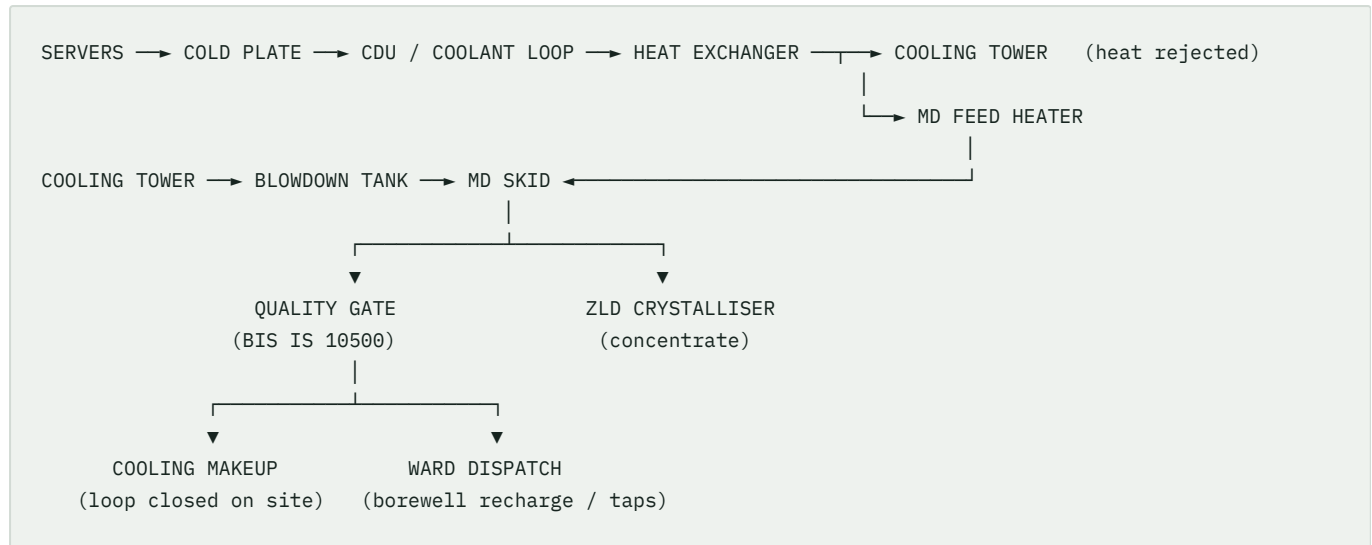
Cooling water splits into two fates, and the distinction matters because operators can report against the more flattering one.

- **Evaporative loss — true consumption.** In an evaporative tower, water absorbs heat and partially evaporates. That fraction is permanently removed from the local watershed. This is why *withdrawal* and *consumption* are different numbers.
- **Blowdown — the wastewater stream.** As water cycles, dissolved solids concentrate until they would scale and corrode the plant, so a portion is drained. Blowdown is *more* concentrated than the intake — more fluoride, more nitrate, more TDS — plus biocides and corrosion inhibitors.

From there a facility has three options: discharge it under permit (the wasteful default), recycle it into the cooling loop (a Greater Noida effluent treatment plant achieved over 95% water savings doing exactly this), or eliminate discharge entirely through Zero Liquid Discharge at high capital cost. NeeruAI's recovery stage targets this stream specifically, because it is the wastewater India's data centres *already produce today* — independent of whether they ever adopt new cooling hardware.

03 The solution — reduce, recover, verify

3.1 The physical chain



3.2 Layer 1 — reduce, at the cold plate

A processor die rejects heat through a thermal interface into a cold plate: a metal block with parallel microchannels. Coolant enters cold, absorbs heat along the channel, leaves hot — and **returns**. It is a sealed recirculating loop, not a consumable.

Contrast with air cooling, which produces low-grade diffuse heat and, with evaporative assist, consumes water continuously. Liquid cooling captures heat at the source, at a higher and more concentrated temperature. That single architectural change does two things at once: it collapses freshwater makeup demand, *and* it upgrades the waste heat into something thermally useful. Layer 1 is therefore not an efficiency footnote — it is what makes Layer 2 physically possible, because air-cooled waste heat is too low-grade to drive distillation.

3.3 Layer 2 — recover, at the membrane

A hydrophobic membrane, typically PTFE with pores around 0.2 μm , separates two channels. Hot concentrated blowdown flows on one side; cool permeate on the other. Because the membrane is hydrophobic, **liquid water cannot pass the pores** — surface tension holds it out. Water *vapour* can. The temperature difference creates a vapour-pressure difference; water evaporates at the hot surface, crosses the pore, and condenses on the cold side.

Fluoride, nitrate and dissolved salts are non-volatile. They do not evaporate, so they physically cannot follow. Rejection here is a phase-change property, not a filtration efficiency being claimed.

WHY MEMBRANE DISTILLATION AND NOT REVERSE OSMOSIS

Reverse osmosis is pressure-driven and runs on electricity — it cannot consume waste heat at all. Specifying RO would collapse the premise that the facility's own rejected heat is the energy input; the system would simply be buying more grid power. Membrane distillation is thermally driven, making it the only mainstream separation technology that actually consumes the byproduct being repurposed.

Temperature polarisation is the detail that separates a working design from a paper one: the membrane surface is never as hot as the bulk feed, because thermal boundary layers form on each side and absorb part of the driving force. The polarisation coefficient τ is the ratio between the ΔT the membrane actually sees and the ΔT an operator measures. Ignoring it overstates flux by roughly 30%, and the prototype models and displays it explicitly.

The concentrate is not hidden. Membrane distillation leaves behind a smaller but more concentrated brine than the blowdown it started from. The architecture names its disposal pathway — a ZLD crystalliser or licensed industrial reuse — because a recovery system that conceals its own residual stream is committing the greenwashing it claims to correct.

3.4 Layer 3 — verify

Inline probes (conductivity/TDS, fluoride ion-selective electrode, nitrate, pH, turbidity) log automatically rather than being self-reported. Every batch is gated against **BIS IS 10500**, the Indian drinking-water specification — deliberately stricter than the industrial-effluent limits a facility would otherwise be judged against. A batch failing any parameter is quarantined and re-treated; it is never released, and the failure remains on the record.

Each batch becomes a document storing the hash of the previous batch's document. Altering any historical record breaks every subsequent hash. This gives tamper-evidence with no blockchain infrastructure — a design that fits inside a hackathon and inside a regulator's audit procedure equally well.

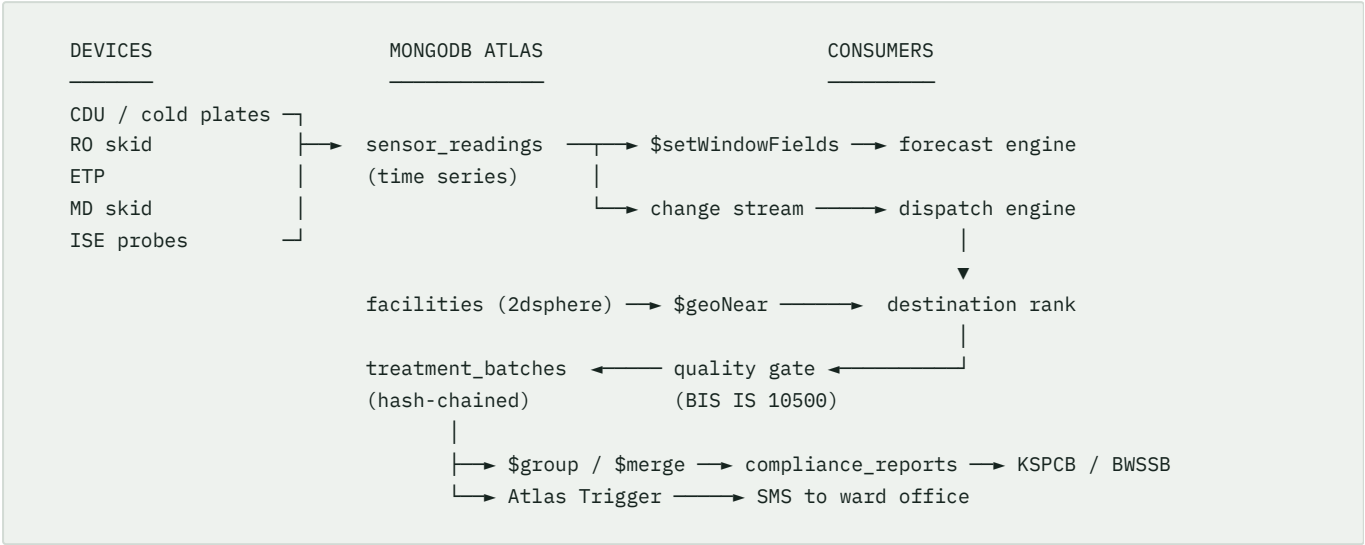
3.5 The coastal variant

For coastal sites — Chennai, Mumbai, Visakhapatnam, Jamnagar — seawater never touches the servers. A closed secondary loop meets seawater only across a heat exchanger, with intake temperature, discharge temperature, ΔT and flow instrumented into the same ledger. The objective is a measured and bounded thermal footprint, not the pretence that none exists.

04 System architecture

Four services sit on one MongoDB deployment. Each corresponds to a section of §05.

SERVICE	RESPONSIBILITY	MONGODB CAPABILITY
Ingestion layer hardware-agnostic	Accepts telemetry from any vendor's CDU, RO skid, ETP or MD unit through one endpoint, preserving each device's native reading shape	Time-series collections; polymorphic documents; schema validation on the envelope only
Forecast engine	Predicts recoverable heat and treatable volume 30 minutes to 6 hours ahead from live utilisation, coolant temperatures, flow, historical load and weather	<code>\$setWindowFields</code> for feature generation directly on the time-series collection
Dispatch engine	Decides when to run the MD skid and where its output goes, prioritising by published groundwater stress	Change streams for reactive triggering; <code>\$geoNear</code> for destination ranking
Verification service	Gates each batch against BIS IS 10500, writes the hash-chained ledger, and generates the statutory water balance	Transactions for chain integrity; aggregation pipeline with <code>\$merge</code> for the compliance API



05 MongoDB data architecture

This section is the substance of the submission. Each capability below is load-bearing: remove it and a named feature of the product stops working. Nothing here is used decoratively.

5.1 Why a document database at all

NeeruAI is four workloads that conventionally live in four different systems:

1. A **high-frequency sensor stream** — thousands of readings per facility per minute, written far more than read, queried almost always by time range.
2. A **polymorphic device-integration layer** — every treatment vendor emits a different reading shape, and the set of vendors grows with every customer.
3. A **geospatial routing problem** — which village, ward or borewell should receive surplus water, ranked by published stress data and distance.
4. An **append-only audit log** with a regulatory reporting API built on top of it.

A relational stack would typically answer this with a time-series database, plus a relational core, plus a PostGIS extension or separate geo service, plus an application-managed audit table — four operational surfaces, four backup strategies, and joins across a network boundary every time the dispatch engine needs to combine a live reading with a facility's stress polygon.

MongoDB collapses all four into one deployment with one query language and one operational surface. The following sub-sections show each one concretely.

5.2 Collection map

COLLECTION	TYPE	ROLE
facilities	standard	Site register: NOC number and cap, declared source, cooling architecture, geometry, district
sensor_readings	time series	All device telemetry. <code>timeField: ts</code> , <code>metaField: source</code>
treatment_batches	standard	The digital water passport and the hash chain. Append-only by role and validator
dispatch_decisions	standard	Every routing decision with a human-readable justification
dispatch_targets	standard	Wards, tap clusters and recharge points with CGWB stress indices and geometry
compliance_reports	materialised	Written by <code>\$merge</code> from the aggregation pipeline; read by regulators
ledger_tips	standard	One document per facility holding the current chain head — see §5.9

5.3 Time-series collections — the ingestion floor

Telemetry is created as a native time-series collection rather than an ordinary one. MongoDB then stores readings in an internal bucketed, columnar format, which is materially better suited to this access pattern than row-per-reading storage: writes are append-heavy, reads are almost always "this facility, this device, this window", and the values within a bucket compress well because consecutive readings are similar.

```
// Telemetry is append-only, time-ordered, and queried by window.
// A native time-series collection gives bucketing, columnar storage and
// automatic expiry without any application-side partitioning logic.
db.createCollection("sensor_readings", {
  timeseries: {
    timeField: "ts",
    metaField: "source", // {facility_id, device_id, device_type, vendor}
    granularity: "seconds"
  },
  expireAfterSeconds: 60 * 60 * 24 * 400 // 400-day hot window; older data → Online Archive
})
```

The `metaField` choice is deliberate. Everything that identifies *where a reading came from* and never changes within a stream goes into `source`; everything that varies reading to reading goes into `metrics`. That is what lets MongoDB group readings from one device into the same bucket and keep bucket counts low.

```
// One reading document as written by the ingestion layer
{
  ts: ISODate("2026-09-08T14:22:03.000Z"),
  source: {
    facility_id: "DC-DVN-03",
    device_id: "CDU-01",
    device_type: "cdu",
    vendor: "Vertiv XDU-450"
  },
  metrics: {
    coolant_supply_C: 32.0,
    coolant_return_C: 45.4,
    flow_L_min: 2184,
    it_load_pct: 74.2,
    heat_reject_kW: 1712,
    junction_temp_C: 78.6
  }
}
```

5.4 Polymorphic documents — what "hardware-agnostic" actually means

This is the single most important schema decision in the project, and the one that would be hardest to replicate in a relational store.

NeeruAI's commercial premise is that it plugs into whatever treatment hardware a facility already runs. But a Vertiv CDU, a domestic RO skid, a municipal ozonation plant and a research membrane distillation module emit *completely different* fields. A fluoride ion-selective electrode reports millivolts and a temperature-compensated concentration; a CDU reports flow, two temperatures and a pump duty; an ETP reports turbidity, BOD proxy and a dosing rate.

Under a rigid schema, onboarding the second vendor requires a migration; the tenth requires either forty mostly-null columns or an unqueryable JSON blob column. Under MongoDB, each device type writes its native shape into the same collection, and the reading is queryable on its own terms immediately:

```
// Three device types, one collection, no migration, all queryable

// Fluoride ion-selective electrode
{ ts: ..., source: { device_type: "ise_probe", vendor: "Hanna HI4110", ... },
  metrics: { electrode_mV: -142.6, temp_C: 27.9, fluoride_mgL: 0.41,
            slope_mV_decade: -57.2, last_cal: ISODate("2026-09-01") } }

// Reverse-osmosis skid
{ ts: ..., source: { device_type: "ro_skid", vendor: "Ion Exchange INDION", ... },
  metrics: { feed_pressure_bar: 14.2, permeate_L_h: 810, reject_L_h: 390,
            conductivity_feed_uS: 2740, conductivity_perm_uS: 41 } }

// Membrane distillation module
{ ts: ..., source: { device_type: "md_skid", vendor: "pilot-DCMD-40", ... },
  metrics: { feed_in_C: 52.4, feed_out_C: 46.1, perm_in_C: 28.0, perm_out_C: 33.7,
            flux_LMH: 18.4, module_area_m2: 40, tmp_kPa: 3.1 } }
```

Validation is applied to the **envelope** only — the fields every device must supply — leaving **metrics** open. This is the practical middle ground between "no schema" and "a schema that blocks the next integration":

```
db.runCommand({
  collMod: "sensor_readings",
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["ts", "source", "metrics"],
    properties: {
      ts: { bsonType: "date" },
      source: { bsonType: "object",
        required: ["facility_id", "device_id", "device_type"],
        properties: {
          facility_id: { bsonType: "string" },
          device_type: { enum: ["cdu", "tower", "ro_skid", "etp",
                                "md_skid", "ise_probe", "flow_meter"] } } },
      metrics: { bsonType: "object" } // intentionally open – this is the integration surface
    } } },
  validationLevel: "moderate"
})
```

5.5 Change streams — reactive dispatch instead of polling

The dispatch engine must respond to a falling coolant return temperature within seconds: below roughly 38 °C the vapour-pressure driving force collapses and continuing to run the MD skid wastes pump energy for negligible permeate. A polling loop either wastes queries or reacts late. A change stream makes the reading itself the trigger.

```
// The new reading PUSHES the scheduler. No cron, no polling interval to tune.
const pipeline = [
  { $match: {
    operationType: "insert",
    "fullDocument.source.device_type": { $in: ["cdu", "md_skid", "ise_probe"] }
  } },
  { $project: { "fullDocument.ts": 1, "fullDocument.source": 1, "fullDocument.metrics": 1 } }
];

const stream = db.collection("sensor_readings")
  .watch(pipeline, { fullDocument: "updateLookup" });

for await (const change of stream) {
  const r = change.fullDocument;

  if (r.source.device_type === "cdu") {
    const feedC = r.metrics.coolant_return_C - HX_LOSS_C;
    await dispatch.evaluateSkid(r.source.facility_id, feedC); // ramp / hold / idle
  }

  if (r.source.device_type === "ise_probe" && r.metrics.fluoride_mgL > BIS.F) {
    await dispatch.holdBatch(r.source.facility_id, {
      reason: `fluoride ${r.metrics.fluoride_mgL} mg/L exceeds BIS limit ${BIS.F}`
    }); // quarantine before a litre leaves the site
  }
}
}
```

Change streams are resumable: the consumer persists a `resumeToken`, so a restarted dispatch service continues from the last processed reading rather than replaying or skipping. For a system whose entire value proposition is a complete, gapless record, that guarantee is not a convenience — it is a correctness requirement.

The same mechanism drives community notification. An Atlas Trigger on `treatment_batches` fires a serverless function that sends an SMS or WhatsApp message to the ward office on each verified delivery — matching how information actually reaches a Devanahalli-type community, rather than assuming anyone there will open an enterprise dashboard.

5.6 Window functions — the forecast engine's feature pipeline

Forecasting recoverable heat needs rolling features: a moving average of rejected heat, its rate of change, and the same-hour value from previous days. These are computed *inside* the database with `$setWindowFields`, directly on the time-series collection, so the model service receives features rather than raw rows — which keeps a very large read off the network entirely.

```

db.sensor_readings.aggregate([
  { $match: {
    "source.facility_id": "DC-DVN-03",
    "source.device_type": "cdu",
    ts: { $gte: new Date(Date.now() - 7*24*3600*1000) }
  } },
  { $setWindowFields: {
    partitionBy: "$source.facility_id",
    sortBy: { ts: 1 },
    output: {
      heat_ma_30m: {
        $avg: "$metrics.heat_reject_kW",
        window: { range: [-1800, 0], unit: "second" } }, // smoothing
      heat_slope_kW_h: {
        $derivative: { input: "$metrics.heat_reject_kW", unit: "hour" },
        window: { documents: [-12, 0] } }, // trend
      load_p95_6h: {
        $percentile: { input: "$metrics.it_load_pct", p: [0.95],
          method: "approximate" },
        window: { range: [-21600, 0], unit: "second" } } // burst headroom
    } } },
  { $project: { _id: 0, ts: 1, heat_ma_30m: 1, heat_slope_kW_h: 1, load_p95_6h: 1,
    heat_now: "$metrics.heat_reject_kW" } }
])

```

`$derivative` is doing real work here: the sign and magnitude of the heat slope is what tells the dispatch engine whether to pre-ramp the skid ahead of an incoming training burst or begin winding it down. In the console, this is the dashed forecast segment that visibly moves *before* the measured line does.

5.7 Geospatial dispatch — ranking who needs the water most

Surplus verified permeate is not sent to whoever asks first. It is ranked against CGWB block-level assessment data, weighted by stress and tempered by haulage distance, with a `2dsphere` index on both facilities and destinations.

```

db.dispatch_targets.createIndex({ geo: "2dsphere" })

db.dispatch_targets.aggregate([
  { $geoNear: {
    near: facility.geo, // GeoJSON Point from the facilities collection
    distanceField: "dist_m",
    maxDistance: 12000, // 12 km tanker-viable radius
    query: { active: true, type: { $in: ["ward_tap", "recharge_shaft"] } },
    spherical: true
  } },
  { $addFields: {
    // stress dominates; distance only breaks ties between comparably stressed blocks
    priority: { $divide: [
      "$cgwb.stage_of_extraction_pct",
      { $add: [1, { $divide: ["$dist_m", 4000] } ] }
    ] } },
  { $sort: { priority: -1 } },
  { $limit: 3 }
])

```

Because facility geometry, destination geometry and the CGWB stress attribute live in the same database as the batch that is about to be dispatched, the entire decision — *is this water clean, how much is spare, and who needs it most* — resolves in one round trip. In a split stack that is a cross-service join on the hot path of every delivery.

5.8 Aggregation — the compliance API a regulator can consume

Karnataka's 2022 Data Centre Policy never required a water report. The Liquid Waste Management Rules, 2024 now impose a reuse floor of 20% by 2027–28 rising to 50% by 2030–31. NeeruAI generates that report from signed batches rather than operator declarations, as a single pipeline materialised with `$merge`:

```
db.treatment_batches.aggregate([
  { $match: { facility_id: facilityId, ts: { $gte: periodStart, $lt: periodEnd } } },

  { $group: {
    _id: { day: { $dateTrunc: { date: "$ts", unit: "day", timezone: "Asia/Kolkata" } } },
    reject_treated_L: { $sum: "$input_L" },
    recovered_L: { $sum: { $cond: [ { $eq: ["$quality", "PASS"] }, "$output_L", 0 ] } },
    quarantined_L: { $sum: { $cond: [ { $eq: ["$quality", "FAIL"] }, "$output_L", 0 ] } },
    to_makeup_L: { $sum: { $cond: [ { $eq: ["$destination_type", "cooling_makeup"] },
      "$output_L", 0 ] } },
    to_community_L: { $sum: { $cond: [ { $eq: ["$destination_type", "community"] },
      "$output_L", 0 ] } },
    heat_recovered_kWh: { $sum: "$heat_used_kWh" },
    batches: { $sum: 1 },
    fluoride_max_mgL: { $max: "$fluoride_mgL" },
    fluoride_p95_mgL: { $percentile: { input: "$fluoride_mgL", p: [0.95],
      method: "approximate" } }
  } },

  { $lookup: { from: "facilities", localField: "facility_id",
    foreignField: "_id", as: "facility" } },
  { $set: { facility: { $first: "$facility" } } },

  { $set: {
    // the number the 2024 Rules actually ask for
    reuse_fraction_pct: { $multiply: [
      { $cond: [ { $gt: ["$reject_treated_L", 0] },
        { $divide: ["$recovered_L", "$reject_treated_L"] }, 0 ] }, 100 ] },
    lwm_2024_status: { $switch: { branches: [
      { case: { $gte: ["$reuse_fraction_pct", 50] }, then: "MEETS_2030_31_TARGET" },
      { case: { $gte: ["$reuse_fraction_pct", 20] }, then: "MEETS_2027_28_TARGET" } ],
      default: "BELOW_FLOOR" } },
    noc_headroom_kld: { $subtract: ["$facility.noc_limit_kld",
      { $divide: ["$freshwater_withdrawn_L", 1000] } ] }
  } },

  { $merge: { into: "compliance_reports",
    on: ["facility_id", "_id.day"],
    whenMatched: "replace", whenNotMatched: "insert" } }
])
```

Because the output is materialised into its own collection, a regulator's dashboard reads pre-computed daily rows rather than re-scanning the ledger. The same pipeline runs unchanged at district or state scope by widening the `$match` and adding a grouping key — which is what makes this a plausible *public* instrument rather than a single operator's report.

5.9 The hash-chained ledger — tamper-evidence without a blockchain

The verification claim is only as good as the log's integrity. Each batch document stores the hash of its predecessor, so modifying any historical record invalidates every hash after it.

The subtlety is concurrency: two batches minted simultaneously must not both claim the same predecessor, or the chain forks. A multi-document transaction against a per-facility `ledger_tips` document serialises the chain head:

```

const session = client.startSession();
await session.withTransaction(async () => {
  // Atomically claim the next height and read the current head
  const tip = await tips.findOneAndUpdate(
    { _id: facilityId },
    { $inc: { height: 1 } },
    { returnDocument: "before", upsert: true, session }
  );

  const doc = {
    ...batch, // litres, chemistry, verdict, destination
    facility_id: facilityId,
    height: (tip?.height ?? 0) + 1,
    prev_hash: tip?.hash ?? GENESIS
  };
  doc.hash = sha256(canonicalJSON(doc)); // deterministic field ordering

  await batches.insertOne(doc, { session });
  await tips.updateOne({ _id: facilityId },
    { $set: { hash: doc.hash } }, { session });
});

```

Append-only behaviour is enforced at two levels rather than trusted to the application. A unique index on `{facility_id, height}` makes a fork impossible to persist, and the writing role is granted `insert` and `find` but not `update` or `remove`:

```

db.treatment_batches.createIndex({ facility_id: 1, height: 1 }, { unique: true })
db.treatment_batches.createIndex({ batch_id: 1 }, { unique: true })

db.createRole({
  role: "ledgerWriter",
  privileges: [ { resource: { db: "neeruui", collection: "treatment_batches" },
    actions: ["insert", "find"] } ], // no update, no remove
  roles: []
})

```

Verification is a single pass any third party can run — the console implements exactly this, and the audit button in the prototype executes it live:

```

db.treatment_batches.aggregate([
  { $match: { facility_id: facilityId } },
  { $sort: { height: 1 } },
  { $setWindowFields: {
    partitionBy: "$facility_id", sortBy: { height: 1 },
    output: { expected_prev: { $shift: { output: "$hash", by: -1,
      default: GENESIS } } } },
  { $match: { $expr: { $ne: ["$prev_hash", "$expected_prev"] } } } // empty result = chain intact
])

```

WHY THIS BEATS AN ACTUAL BLOCKCHAIN HERE

The property a regulator needs is *tamper-evidence*, not decentralised consensus — there is no adversarial multi-party write problem to solve, because the operator is the only writer and the regulator is the auditor. A hash chain in a document field delivers the needed property with no consensus overhead, no token, no separate infrastructure to operate, and a verification procedure a state pollution control board's IT team can run in one query.

5.10 Index strategy

COLLECTION	INDEX	SERVES
facilities	{ geo: "2dsphere" }	Dispatch radius search from a facility
facilities	{ district: 1, stage: -1 }	Regulator's district-level fleet view
dispatch_targets	{ geo: "2dsphere" }	<code>\$geoNear</code> destination ranking (§5.7)
sensor_readings	implicit ts + source	Created automatically by the time-series collection
sensor_readings	{ "source.device_type": 1, ts: -1 }	Per-device-class windows for the forecast pipeline
treatment_batches	{ facility_id: 1, height: 1 } <code>UNIQUE</code>	Chain integrity — makes a fork unpersistable
treatment_batches	{ facility_id: 1, ts: -1 }	Ledger view and compliance <code>\$match</code>
treatment_batches	{ quality: 1, ts: -1 }	Quarantine audit — "show me every failed batch this quarter"
dispatch_decisions	{ facility_id: 1, ts: -1 }	Decision replay when a routing choice is challenged

5.11 Scale, retention and cost shape

The arithmetic that sizes the deployment:

QUANTITY	VALUE	DERIVATION
Probes per facility	~8	CDU pair, tower, flow meter, conductivity, fluoride ISE, nitrate, pH
Readings per facility per day	691,200	$8 \times 1 \text{ Hz} \times 86,400 \text{ s}$
Readings at 100 facilities	69.1 M / day	Roughly 800 writes per second sustained
Batch documents	~12 / facility / day	One per two-hour treatment cycle
Dispatch decisions	~12 / facility / day	One per batch
Compliance rows	1 / facility / day	Materialised by <code>\$merge</code>

The shape that matters is the ratio: telemetry outnumbers ledger documents by roughly **five orders of magnitude**. Those two workloads have opposite requirements — telemetry is write-heavy, compressible, and expendable after its retention window; the ledger is tiny, immutable, and must be preserved indefinitely. Putting them in one deployment but different collection *types* lets each get the treatment it needs:

- **Telemetry** — time-series collection with bucketing and columnar compression, a 400-day `expireAfterSeconds` hot window, and Atlas Online Archive beyond that for the cold tail a regulator might subpoena years later.
- **Ledger** — a standard collection, never expired, with the unique index and restricted role from §5.9.
- **Reports** — materialised daily rows, so the regulator-facing read path never touches the ledger scan.

5.12 Atlas platform services used

SERVICE	USE IN NEERUAI
Atlas Triggers / Functions	Fired from change streams to push SMS/WhatsApp delivery notices to ward offices, and to run the nightly compliance <code>\$merge</code> . Serverless, so there is no scheduler to operate at a site with no IT staff.
Atlas Charts	The regulator-facing view — a KSPCB or BWSSB officer gets a dashboard over <code>compliance_reports</code> without NeeruAI building or hosting a separate frontend for each board.
Atlas Online Archive	Automatic tiering of telemetry past the hot window, keeping the queryable surface intact for historical audit at archive cost.
Atlas Vector Search PLANNED	Semantic retrieval over environmental clearances, NOC conditions and CGWB reports, so an inspector can ask: <i>"which facilities declared municipal supply but show groundwater-consistent conductivity signatures?"</i> — joining unstructured regulatory text to structured telemetry in one system.
Queryable Encryption PLANNED	NOC numbers and commercially sensitive consumption figures encrypted in use, so a facility can participate in a shared regulator view without exposing competitive data to other operators on the platform.

5.13 Why not a relational stack — direct comparison

REQUIREMENT	RELATIONAL APPROACH	MONGODB APPROACH
Ingest 69 M readings/day	Separate time-series database, or manual partitioning and a rollup job	Native time-series collection with automatic bucketing and expiry
Onboard vendor #11	Schema migration, or 40 nullable columns, or an opaque JSON column	Write the device's native shape; it is queryable immediately
React to a temperature drop	Polling loop, or a trigger plus a message broker	Change stream with a resume token
Rolling forecast features	Window functions, but on rows shipped from a second system	<code>\$setWindowFields</code> on the same collection, in place
Rank destinations by stress	PostGIS extension or an external geo service; cross-system join	<code>\$geoNear</code> in the same query as the batch data
Statutory report	ETL into a warehouse; a third system to operate	One pipeline with <code>\$merge</code> into a materialised collection
Tamper-evident log	Application-enforced audit table, or a blockchain deployment	Hash chain in a document field, unique index, restricted role

The count that matters: **four operational surfaces reduced to one**. For a product whose customers include a municipal water board and a state pollution control board — organisations with limited platform engineering capacity — that reduction is not developer convenience. It is whether the system can be operated at all after handover.

06 The prototype

A six-page operations console. Every figure below is a screenshot of the running simulation, not a mockup — and every number in every panel derives from one shared physics chain, so a control changed on one page moves every downstream page consistently.

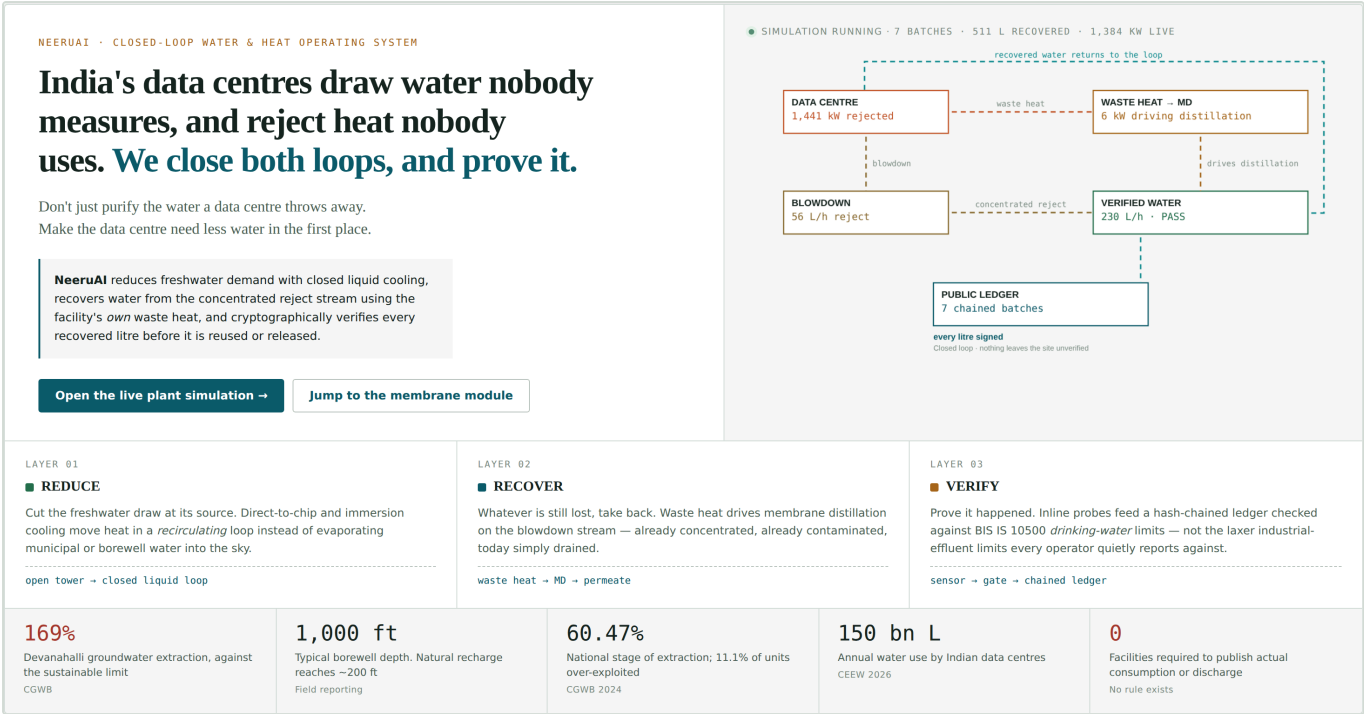


Figure 1 — Overview. The thesis page: the three layers, the five figures that establish the problem, and a live loop diagram whose values come from the running simulation. The final figure — zero facilities required to publish what they use — is the business case.

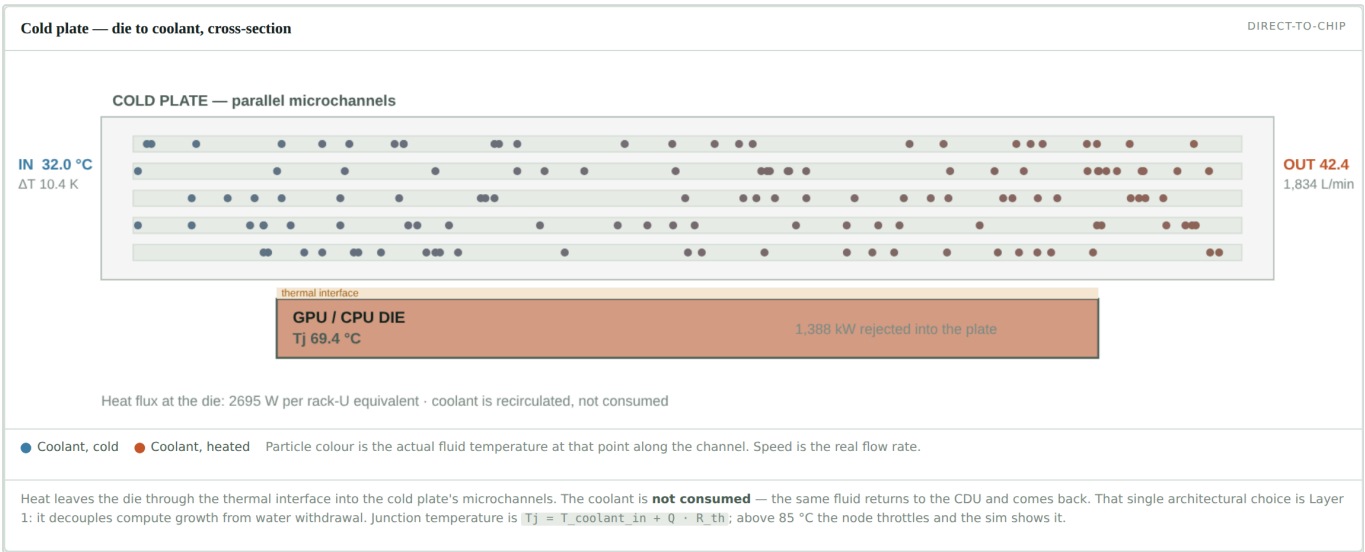


Figure 2 — Stage 01, cold plate cross-section. Coolant particles travel the microchannels; their colour is the real fluid temperature at that point, interpolated between the live supply and return values, and their speed is the real flow rate. Junction temperature is computed from coolant inlet plus loop rise, and nodes throttle above 85 °C.

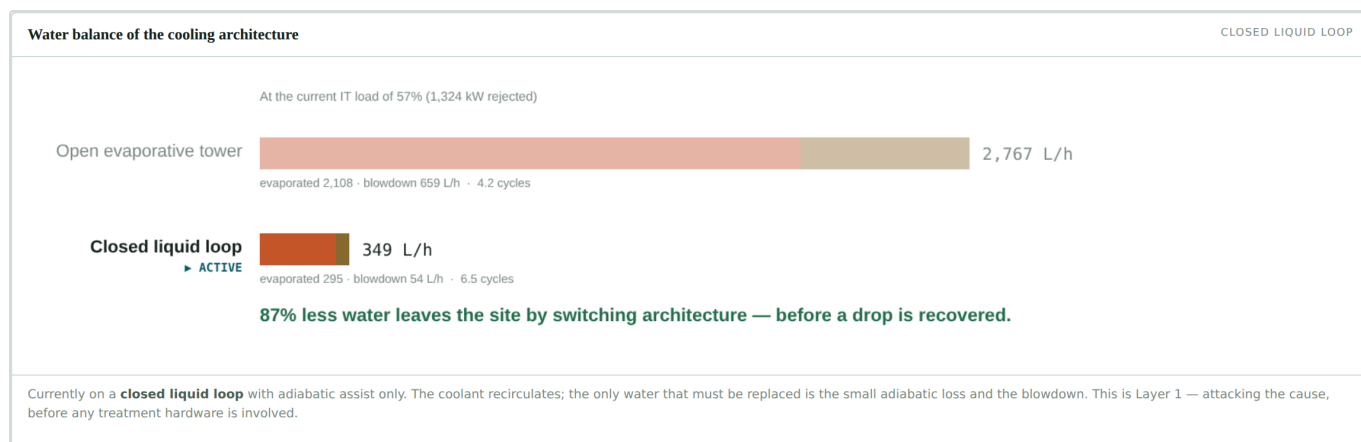


Figure 3 — Stage 01, architecture comparison. Water leaving the site per hour at the current heat load, split into evaporation and blowdown, for open evaporative tower against closed liquid loop. Toggling the architecture live is the demonstration of Layer 1: roughly 87% less water leaves the site before any recovery has occurred.

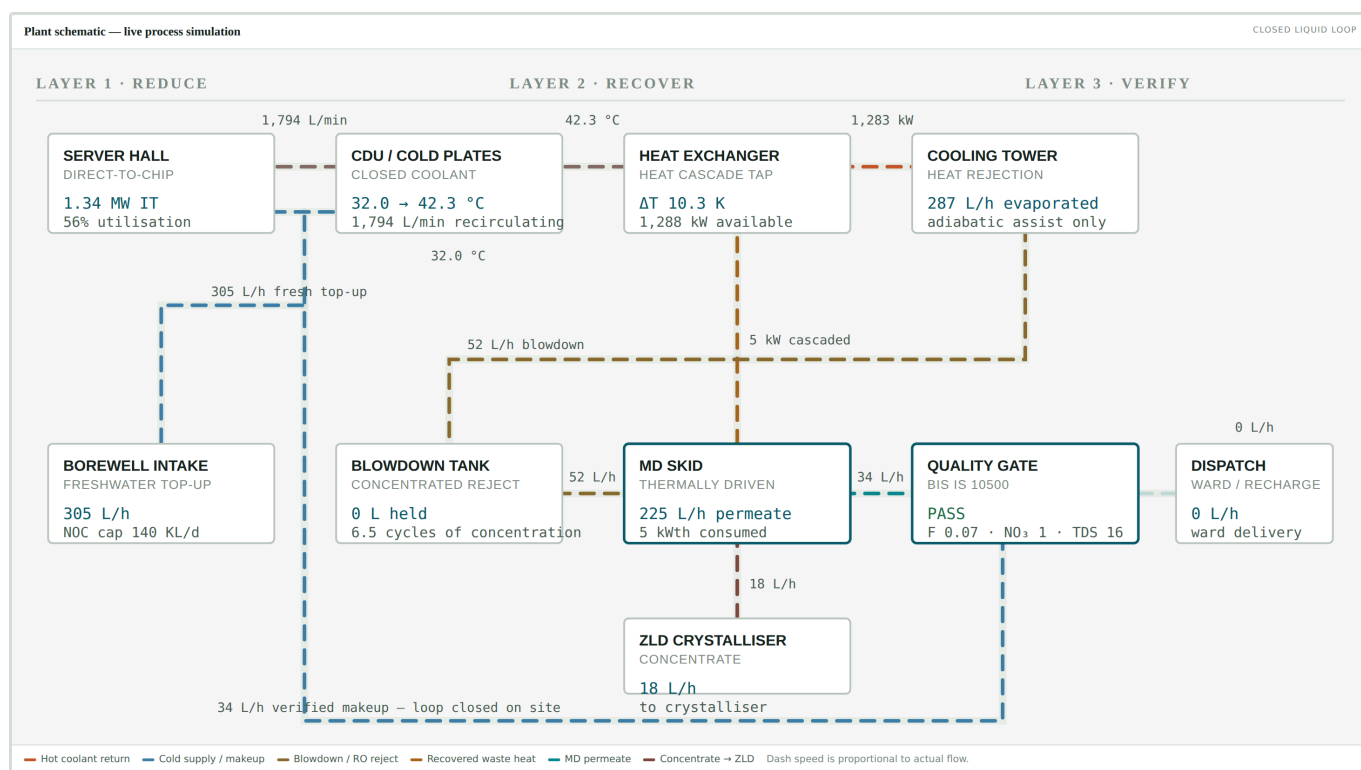


Figure 4 — Stage 02, plant schematic. The whole plant on one sheet, banded by layer. Every pipe animates at a speed proportional to its actual flow and fades when a stream goes idle; hot coolant lines shift colour with return temperature. The long return path at the bottom is the closed loop — makeup served from verified permeate before any borewell top-up.

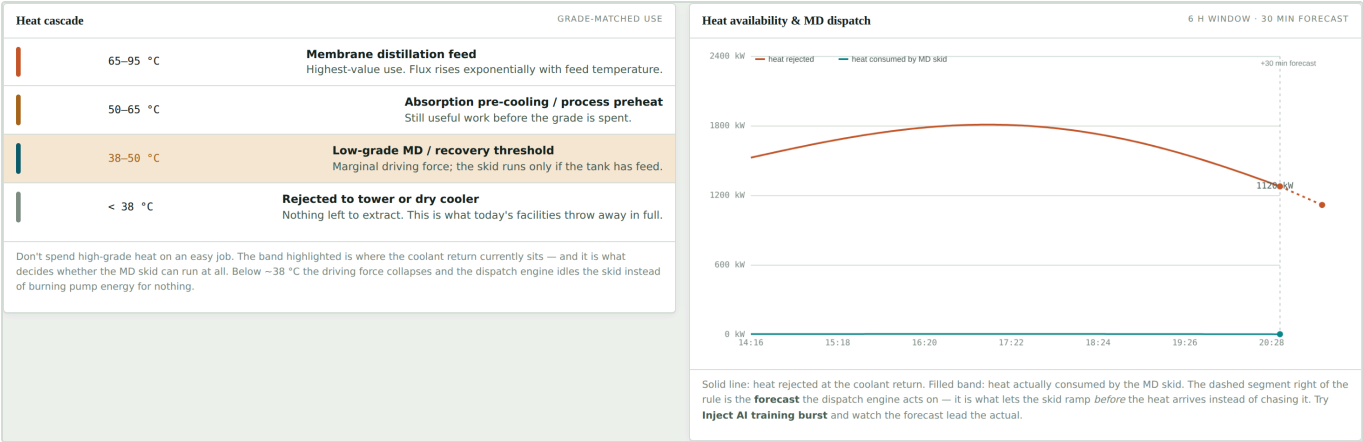


Figure 5 — Stage 02, heat cascade and dispatch forecast. Left: temperature bands with the current coolant return highlighted — below 38 °C the driving force collapses and the dispatch engine idles the skid. Right: heat rejected against heat consumed by the MD skid, with the dashed +30 minute forecast the dispatch engine is acting on. The gap between the two lines is the addressable opportunity, live.

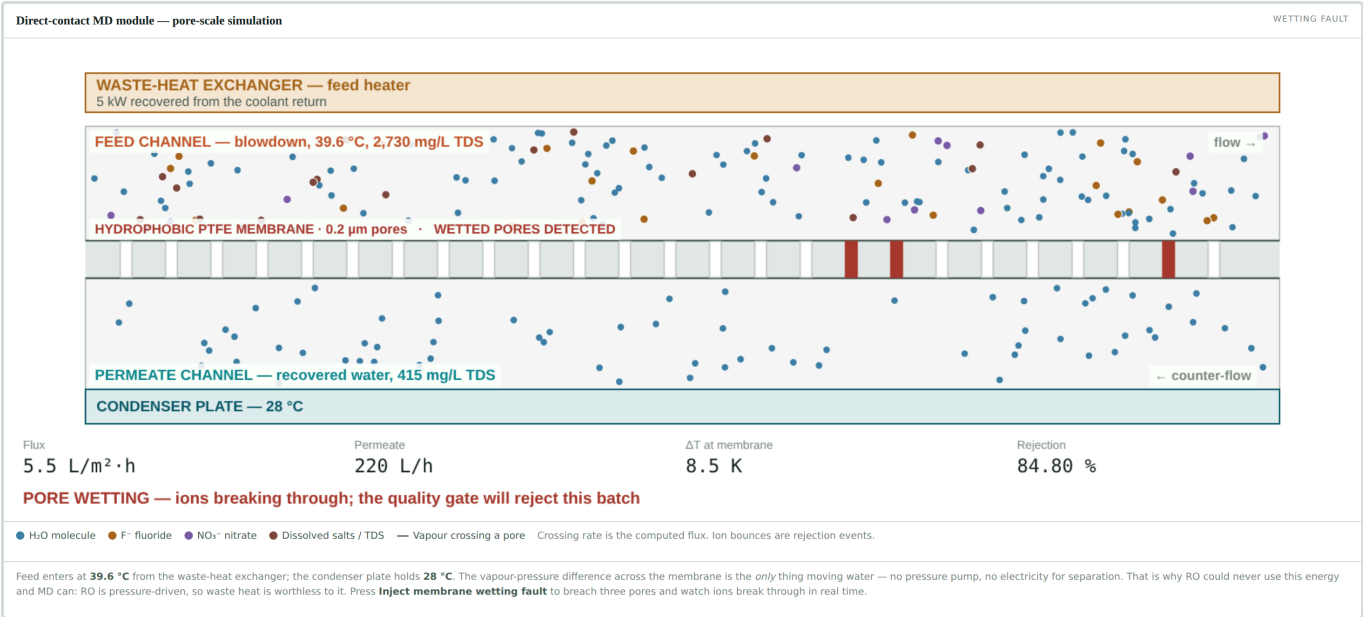


Figure 6 — Stage 03, membrane distillation at pore scale. Feed channel with hot concentrated blowdown above, hydrophobic PTFE membrane with 0.2 µm pores, permeate channel and condenser plate below. Dashed vertical transits are water molecules crossing as vapour; coloured particles bouncing off the membrane are fluoride, nitrate and salt rejection events. Shown here **DURING A WETTING FAULT** — three pores breached, ions convecting through, rejection collapsed from 99.4% to 84.8%.

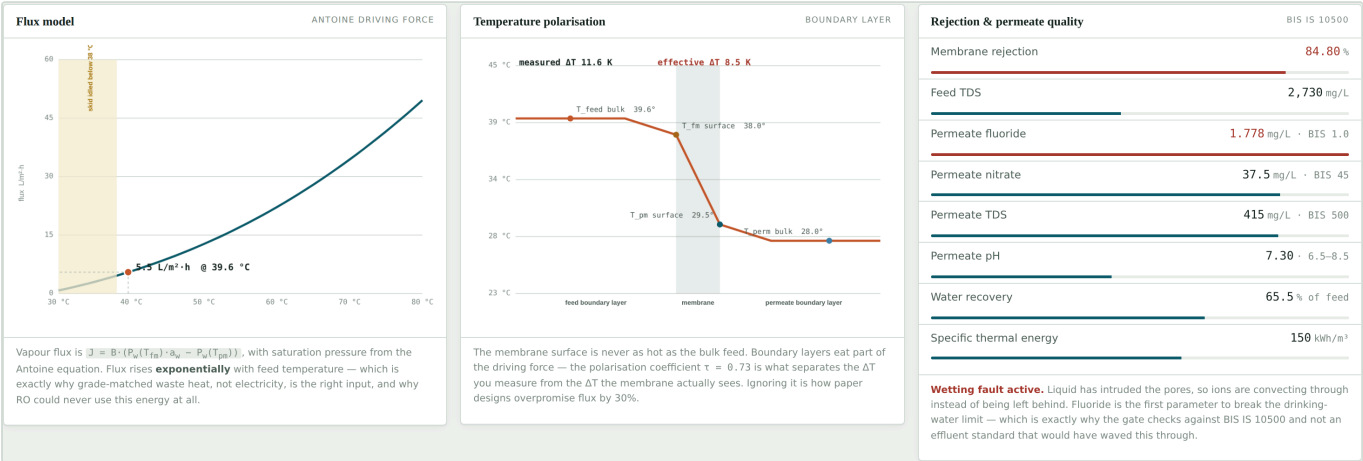


Figure 7 — Stage 03, the model behind the animation. Left: flux against feed temperature from the Antoine relation, with the live operating point and the shaded region below 38 °C where the skid idles. Centre: the temperature profile across both boundary layers, separating measured ΔT from the effective ΔT the membrane actually sees. Right: permeate chemistry against BIS IS 10500, with fluoride in breach during the injected fault.

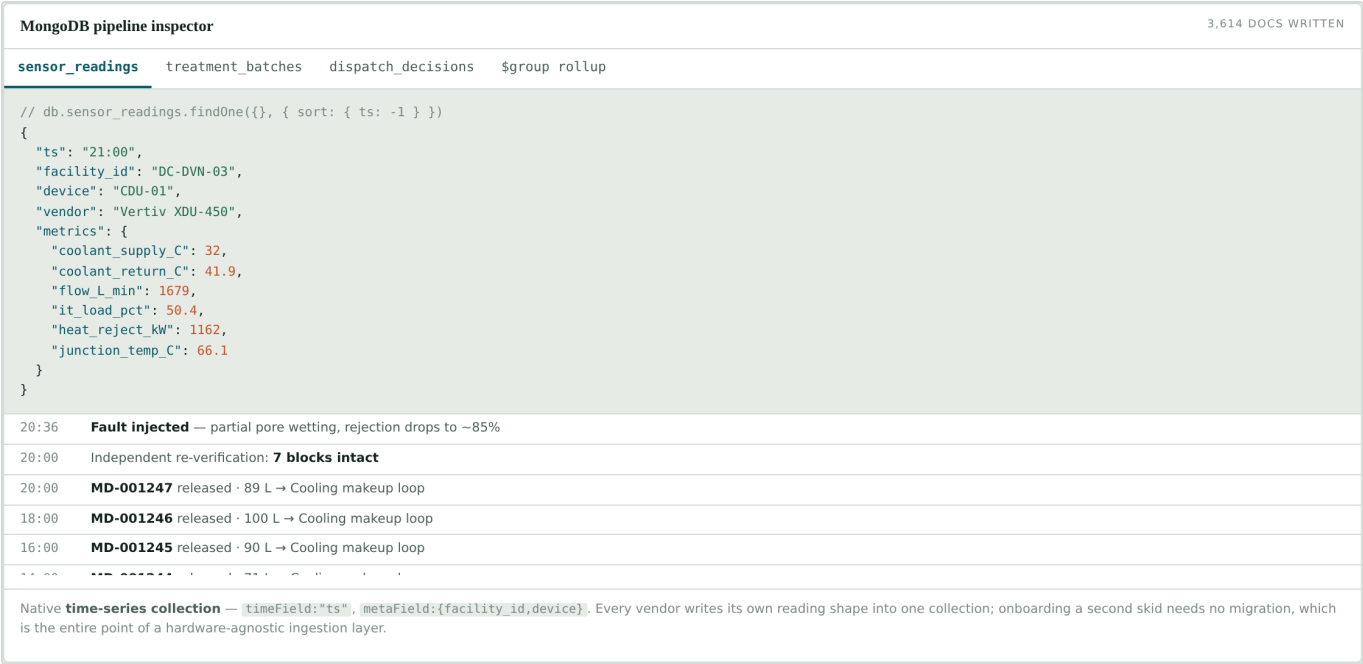


Figure 8 — Stage 04, MongoDB pipeline inspector. The actual documents being written, with a running write count and the change-stream event feed beneath. Four tabs: the time-series reading with its vendor-specific payload, the hash-chained batch, the change-stream-written dispatch decision with its human-readable justification, and the `$group` rollup that produces the compliance report.

Batch ledger		HASH CHAINED
MD-001247	PASS	20:00
In: 132 L, out: 89 L, 67.7% 13.4 kWh F 0.068		
→ Cooling makeup loop		
7d531b87fe6bc155 • 38f18ca8ef1f4c2a		
MD-001246	PASS	18:00
In: 146 L, out: 100 L, 68.5% 14.9 kWh F 0.068		
→ Cooling makeup loop		
c2a731f25740f870 • 7d531b87fe6bc155		
MD-001245	PASS	16:00
In: 123 L, out: 98 L, 67.7% 13.5 kWh F 0.068		
→ Cooling makeup loop		
633c0baf5757086d • c2a731f25740f870		
MD-001244	PASS	14:00
In: 100 L, out: 71 L, 66.2% 10.7 kWh F 0.068		
→ Cooling makeup loop		
17396c0b370306da • 633c0baf5757086d		
MD-001243	PASS	12:00
In: 88 L, out: 57 L, 64.7% 9.6 kWh F 0.068		
→ Cooling makeup loop		
320c0baf5757086d • 17396c0b370306da		

Figure 9 — The ledger. Each row shows litres in and out, recovery, heat used, fluoride, destination, and the prev_hash ► hash link. Failed batches are shaded and routed to quarantine — and they stay on the record. A log that only records successes is not a verification system.

Quality gate

FAIL

Fluoride

1.778 / 1.0 mg/L

Nitrate

37.5 / 45 mg/L

TDS

415 / 500 mg/L

pH

7.30 / 6.5–8.5

Limits are **BIS IS 10500 drinking water**, deliberately stricter than the industrial-effluent limits a facility would otherwise be judged against. A batch that fails any parameter is routed to quarantine and re-treated. It is never released, and the failure stays on the public ledger.

Digital water passport

LATEST BATCH

Batch

MD-001247

Facility

DC-DVN-03

NOC

CGWB/KA/2024/0881

Source

cooling-blowdown

Feed temp

40.2 °C

In → out

132 L → 89 L (67.7%)

Heat used

13.4 kWh recovered

Fluoride

0.068 / 1 mg/L

Nitrate

1.4 / 45 mg/L

TDS

16 / 500 mg/L

Verdict

PASS • BIS IS 10500

Destination

Cooling makeup loop

prev_hash

7d531b87fe6bc155

hash

38f18ca8ef1f4c2a

Figure 10 — Quality gate and water passport. The full record for one batch: source, feed temperature, litres, heat used, all three contaminants against their limits, verdict, destination, and both hashes. It answers five questions checkably that no Indian regulator can answer about any facility today.

Dispatch — CGWB stress-ranked destinations				2DSphere · 12 KM		Statutory water balance			Auto-generated	
Destination	Stage of extraction	Rank	Delivered today				Parameter	Today	Unit	
Cooling makeup loop	—	—	526 L				Freshwater withdrawn	4,792	L	
Chikkajala ward tap cluster	Over-exploited · 169%	1	0 L				Evaporative loss (consumed)	4,500	L	
Bettakote borewell recharge	Over-exploited · 152%	2	0 L				Blowdown generated	818	L	
Vishwanathapura tanker offset	Critical · 128%	3	0 L				Reject treated by MD	818	L	
Quarantine — re-treat	Failed BIS gate	—	17 L				Water recovered (verified)	526	L	
Ranking is a sgear against CGWB block-level assessment polygons, re-scored on every change-stream event. Cooling makeup is served first — the loop is closed before anything leaves the fence — and only surplus verified permeate is dispatched outward. Ward offices get an SMS per delivery; no dashboard login required, because that is how information actually reaches a Devanahalli-type community.							Returned to cooling makeup	526	L	
							Delivered to community	0	L	
							Concentrate to ZLD	275	L	
							Waste heat consumed	81	kWh	
							Discharged untreated	0	L	
				Reuse fraction			LWM 2024 · 2030-31 Target Met	64.3	%	
Rendered in the shape KSPCB / BWSSB would consume. The Liquid Waste Management Rules, 2024 set a reuse floor of 20% by 2027-28 rising to 50% by 2030-31 for large water users; Karnataka's 2022 Data Centre Policy required no water report at all. Every figure traces to a signed batch — nothing here is operator-declared.										

Figure 11 — Stage 05, dispatch and statutory compliance. Left: destinations ranked by CGWB stage of extraction, with cooling makeup served first so the loop closes before anything leaves the fence. Right: the water balance in the shape KSPCB or BWSSB would consume, with the reuse fraction scored against the Liquid Waste Management Rules 2024 targets. Note that *withdrawn* and *consumed* are separate rows — the distinction most corporate reporting blurs.

Interactive demonstrations built into the console

CONTROL	WHAT IT DEMONSTRATES
Cooling architecture toggle	Layer 1 in one click — evaporation factor, cycles of concentration and supply temperature all change, and the water balance collapses
Inject AI training burst	Load jumps 30 points; the forecast on the chart moves before the measured line does, showing why a dispatch layer is needed
Inject membrane wetting fault	Three pores breach, ions break through, fluoride crosses 1.0 mg/L, and the gate quarantines the batch rather than delivering it
Re-verify hash chain	Recomputes every hash and every <code>prev_hash</code> link independently — the audit a regulator would run
Facility selector	Four sites, each making a different argument: the base case, a declared-source mismatch, a well-run counterexample, and a clearance that names no water source at all

07 Engineering model

Nothing in the console is a canned animation. One physics chain drives all six pages — which is why changing a control on one page moves every other page consistently.

QUANTITY	MODEL	NOTE
IT power	$P = MW_IT \times load$	Load follows a diurnal curve or manual override, smoothed so it ramps
Heat rejected	$Q = 0.96 \times P$	Essentially all IT power becomes heat
Coolant flow	$620 + 2100 \times load \text{ L/min}$	Pump scales with demand
Loop ΔT	$\Delta T = Q \cdot 60 / (\dot{m} \cdot c_p)$	$c_p = 4.18 \text{ kJ/kg}\cdot\text{K}$; return = supply + ΔT
Junction temperature	$T_j = T_{supply} + \Delta T + 38 \cdot load + 5$	Throttles above 85 °C
Evaporation	$1.593 \times Q \times f \text{ L/h}$	From latent heat $\approx 2260 \text{ kJ/kg}$. $f = 1.0$ open tower, 0.14 closed loop
Blowdown	$evaporation / (CoC - 1)$	$CoC = 4.2$ tower, 6.5 closed loop
MD feed temperature	$T_f = T_{return} - 2.5 \text{ }^\circ\text{C}$	Heat exchanger loss; permeate side held at 28 °C
Polarisation coefficient	$\tau = 0.78 - 0.004 \cdot (T_f - T_p)$	Bounded 0.55–0.82; boundary layers worsen as ΔT grows
Membrane surface temps	$T_{fm} = T_f - (1-\tau)/2 \cdot \Delta T$ $T_{pm} = T_p + (1-\tau)/2 \cdot \Delta T$	What the membrane actually sees
Vapour pressure	Antoine equation	$\log_{10}P = 8.07131 - 1730.63/(233.426+T)$, mmHg $\rightarrow$ Pa
Flux	$J = B \cdot (P_w(T_{fm}) \cdot a_w - P_w(T_{pm}))$	$B = 2.2 \times 10^{-3} \text{ L}/(\text{m}^2 \cdot \text{h} \cdot \text{Pa})$, area 40 m ² ; a_w reduced by feed TDS
Thermal demand	150 kWh per m ³ permeate	Typical MD specific energy
Rejection	99.42% nominal / 84.8% wetted	Degrades slightly above 52 °C feed
Permeate quality	$C_{perm} = C_{intake} \times CoC \times (1 - R)$	Concentration first, then rejection — this is why fluoride breaks first

08 Roadmap to production

PHASE	DELIVERABLE	MONGODB WORK
Now prototype	Six-page console, full physics simulation, in-browser ledger and audit	Schema and pipeline design validated against the simulated workload
Phase 1 single site	Live ingestion from one facility's CDU and existing ETP; read-only regulator view	Atlas cluster; time-series collection; change-stream dispatch service; Atlas Charts view for the board
Phase 2 verification pilot	Inline fluoride and conductivity probes on the permeate line; first signed batches; SMS to one ward office	Ledger transactions and unique index; restricted <code>ledgerWriter</code> role; Atlas Trigger for notifications
Phase 3 fleet	Multi-facility, multi-vendor onboarding; district-level regulator dashboard	Polymorphic ingestion proven across vendors; <code>\$merge</code> compliance pipeline at district scope; Online Archive tiering
Phase 4 regulatory product	Natural-language regulatory query over clearances and telemetry; encrypted multi-tenant regulator view	Atlas Vector Search over clearance documents; Queryable Encryption on commercially sensitive fields

What a real deployment must validate

- **Membrane rejection** is feed- and membrane-specific. The model uses literature values; a deployment must measure actual rejection against its own blowdown chemistry rather than quoting a brochure figure.
- **Flux coefficient B** and module area must be characterised for the specific membrane and module geometry procured.
- **Probe calibration drift** — a fluoride ion-selective electrode needs scheduled recalibration, and the ledger should record calibration state alongside each reading so a batch's provenance includes the instrument's condition. The schema already carries `last_cal` on ISE readings for exactly this.
- **Concentrate disposal** must be contracted before commissioning, not after.

09 Sources & honest limitations

WHAT IS MODELLED, WHAT IS MEASURED

Modelled from first principles and standards: the heat balance, loop ΔT , evaporation from latent heat, cycles of concentration, the Antoine vapour-pressure relation, temperature polarisation, the flux model, and the concentration-then-rejection chain producing permeate chemistry. The regulatory limits, extraction figures and reuse targets are as published. The hash chain is genuinely computed and genuinely re-verifiable.

Simulated: the telemetry itself is synthetic, generated by the model rather than read from a facility. The four facilities are composites assembled from published reporting, not live customers. Membrane coefficients are typical literature values, not measurements from a specific procured module. The prototype states this in its own footer.

The plant is simulated, but it is not faked — every number on screen is computed from a physical model, which is why a control changed on one page moves every other page consistently. What a real deployment requires is calibration, not redesign.

Sources

Central Ground Water Board — Annual Groundwater Quality Report and Dynamic Groundwater Resource Assessment, 2024 · Council on Energy, Environment and Water (CEEW) — data centre water and electricity analysis, 2026 · Down To Earth — *India's Digital Thirst* (two parts, November 2025) and *Thirsty Data Centres, Thin Regulation* (August 2026) · Mongabay India — *India Bets on Data Centres Even as Water, Energy-Use Concerns Mount* (May 2026) · Scroll.in (May 2026) · Newslandry (April 2026) · OpenCity (May 2026) · The Better India — Devanahalli water reuse project (October 2024) · Nxtra by Airtel — ESG Report 2025 · Telangana Today / Domain-b — Andhra Pradesh ministerial statements on Visakhapatnam data centre water sourcing (August 2026) · Springer, *Environmental Geochemistry and Health* — Global Groundwater Contamination by Geogenic Fluoride (2026) · Springer, *Applied Water Science* — fluoride contamination in hard-rock aquifers, Siddipet, Telangana · ScienceDirect — *Fluoride Removal from Brackish Groundwater by Direct Contact Membrane Distillation* · ScienceDirect / PubMed — *Removal of Fluoride in Membrane-Based Water and Wastewater Treatment Technologies: Performance Review* · Government of India — Liquid Waste Management Rules, 2024 · Envirotech Online — India 2025 Water Pollution Consent Guidelines · Susbio — Zero Liquid Discharge Implementation in India · Government of Karnataka — Data Centre Policy 2022 · Bureau of Indian Standards — IS 10500 Drinking Water Specification · World Health Organization — Guidelines for Drinking-water Quality.

CLOSING

India is about to build a great deal of data centre capacity in places that cannot spare the water. The engineering to use less of it, and to recover most of what is still lost, largely exists. What does not exist is any mechanism to establish whether a facility is actually doing it. NeeruAI is that mechanism — and MongoDB is what makes it one system instead of four.